



5. Decision Trees and Random Forests

Overview

In this chapter, we'll shift our focus to another type of machine learning model that has taken data science by storm in recent years: tree-based models. In this chapter, after learning about trees individually, you'll then learn how models made up of many trees, called random forests, can improve the overfitting associated with individual trees. After reading this chapter, you will be able to train decision trees for machine learning purposes, visualize trained decision trees, and train random forests and visualize the results.

Introduction

In the last two chapters, we have gained a thorough understanding of the workings of logistic regression. We have also gotten a lot of experience with using the scikit-learn package in Python to create logistic regression models.

In this chapter, we will introduce a powerful type of predictive model that takes a completely different approach from the logistic regression model: **decision trees**. Decision trees and the models based on them are some of the most performant models available today for general machine learning applications. The concept of using a tree process to make decisions is simple, and therefore, decision tree models are easy to interpret. However, a common criticism of decision trees is that they overfit to the training data. In order to remedy this issue, researchers have developed **ensemble methods**, such as **random**

forests, that combine many decision trees to work together and make better predictions than any individual tree could.

We will see that decision trees and random forests can improve the quality of the predictive modeling of the case study data beyond what we have achieved so far with logistic regression.

Decision Trees

Decision trees and the machine learning models that are based on them, in particular, **random forests** and **gradient boosted trees**, are fundamentally different types of models than Generalized Linear Models (GLMs), such as logistic regression. GLMs are rooted in the theories of classical statistics, which have a long history. The mathematics behind linear regression was originally developed at the beginning of the 19th century, by Legendre and Gauss. Because of this, the normal distribution is also known as the Gaussian distribution.

In contrast, while the idea of using a tree process to make decisions is relatively simple, the popularity of decision trees as mathematical models has come about more recently. The mathematical procedures that we currently use for formulating decision trees in the context of predictive modeling were published in the 1980s. The reason for this more recent development is that the methods used to grow decision trees rely on computational power – that is, the ability to crunch a lot of numbers quickly. We take such capabilities for granted nowadays, but they weren't widely available until more recently in the history of mathematics.

So, what is meant by a decision tree? We can illustrate the basic concept using a practical example. Imagine that you are considering whether or not to venture outdoors on a certain day. The only information you will base your decision on involves the weather and, in particular, whether the sun is shining and how warm it is. If it is

sunny, your tolerance for cool temperatures is increased, and you will go outside if the temperature is at least 10 °C.

However, if it's cloudy, you require somewhat warmer temperatures and will only go outside if the temperature is 15 °C or more. Your decision-making process could be represented by the following tree:

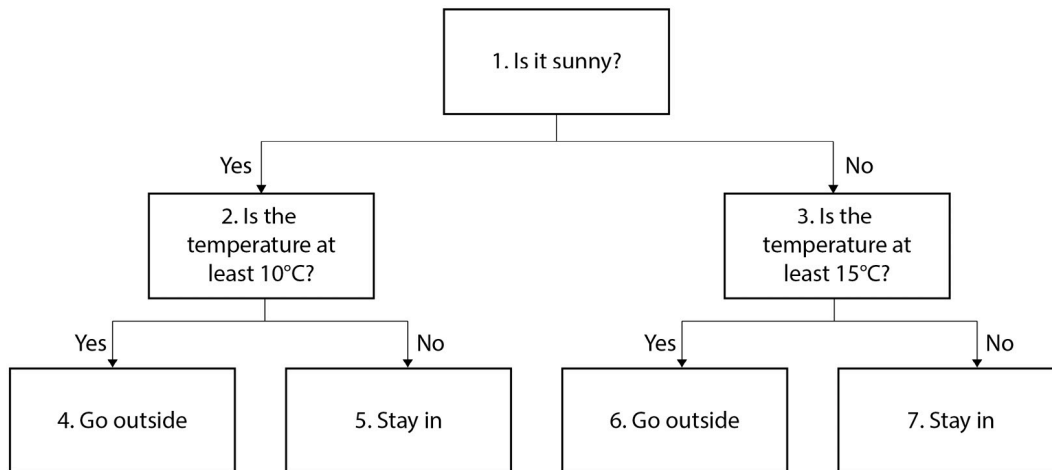


Figure 5.1: A decision tree for deciding whether to go outside given the weather

As you can see, decision trees have an intuitive structure and mimic the way that logical decisions might be made by humans. Therefore, they are a highly **interpretable** type of mathematical model, which can be a particularly desirable property depending on the audience. For example, the client for a data science project may be especially interested in a clear understanding of how a model works. Decision trees are a good way of delivering on this requirement, as long as their performance is sufficient.

The Terminology of Decision Trees and Connections to Machine Learning

Looking at the tree in *Figure 5.1*, we can begin to become familiar with some of the terminology of decision trees. Because there are two levels of decisions being made, based on cloud conditions at the first level and temperature at the second level, we say that this decision tree has

a **depth** of two. Here, both **nodes** at the second level are temperature-based decisions, but the kinds of decisions could be different within a level; for example, we could base our decision on whether or not it was raining if it was not sunny.

In the context of machine learning, the quantities that are used to make decisions at the nodes (in other words, to **split** the nodes) are the features. The features in the example in *Figure 5.1* are a binary categorical feature for whether it's sunny, and a continuous feature for temperature. While we have only illustrated each feature being used once in a given branch of the tree, the same feature could be used multiple times in a branch. For example, we may choose to go outside on a sunny day with a temperature of at least 10 °C, but not if it were more than 40 °C – that's too hot! In this case, node 4 of *Figure 5.1* would be split on the condition "Is the temperature greater than 40 °C?" where "stay in" is the outcome if the answer is "yes," but "go outside" is the outcome if the answer is "no," meaning that the temperature is between 10 °C and 40 °C. Decision trees are therefore able to capture non-linear effects of the features, as opposed to a linear relationship that might assume that the hotter it was, the more likely we would be to go outside, regardless of how hot it was.

Consider the way that trees are typically represented, such as in *Figure 5.1*. The branches grow downward based on the binary decisions that can split the nodes into two more nodes. These binary decisions can be thought of as "if, then" rules. In other words, if a certain condition is met, do this, otherwise, do something else. The decision being made in our example tree is analogous to the concept of the response variable in machine learning. If we made a decision tree for the case study problem of credit default, the decisions would instead be predictions of the binary response values, which are "this account defaults" or "this account doesn't default." A tree that answers a binary yes/no type of question is a **classification tree**. However, decision trees are quite versatile and can also be used for multiclass classification and regression.

The terminal nodes at the bottom of the tree are called **leaves**, or leaf nodes. In our example, the leaves are the final decisions as to whether to go outside or stay in. There are four leaves on our tree, although you can imagine that if the tree only had a depth of one, where we made our decision based only on cloud conditions, there would be two leaves; and nodes 2 and 3 in *Figure 5.1* would be leaf nodes with "go outside" and "stay in" as the decisions, respectively.

In our example, every node at every level before the final level is split. This is not strictly necessary as you may go outside on any sunny day, regardless of the temperature. In this case, node 2 will not be split, so this branch of the tree will end on the first level with a "yes" decision. Your decision on cloudy days, however, may involve temperature, meaning this branch can extend to a further level. In the case that every node before the final level is split, consider how quickly the number of leaves grows with the number of levels.

For example, what would happen if we grew the decision tree in *Figure 5.1* down through an additional level, perhaps with a wind speed feature, to factor in wind chill for the four combinations of cloud conditions and temperature. Each of the four nodes that are now leaves, nodes numbered from four to seven in *Figure 5.1*, would be split into two more leaf nodes, based on wind speed in each case. Then, there would be $4 \times 2 = 8$ leaf nodes. In general, it should be clear that in a tree with n levels, where every node before the final level is split, there will be 2^n leaf nodes. This is important to bear in mind as **maximum depth** is one of the hyperparameters that you can set for a decision tree classifier in scikit-learn. We'll now explore this in the following exercise.

Exercise 5.01: A Decision Tree in Scikit-Learn

In this exercise, we will use the case study data to grow a decision tree, where we specify the maximum depth. We'll also use some handy

functionality to visualize the decision tree, in the form of the **graphviz** package. Perform the following steps to complete the exercise:

Note

The Jupyter notebook for this exercise can be found at

<https://packt.link/IUt7d>. Before you begin the exercise, please ensure that you have followed the instructions in the Preface regarding setting up your environment and importing the necessary libraries.

1. Load several of the packages that we've been using, and an additional one, **graphviz**, so that we can visualize decision trees:

```
import numpy as np #numerical computation
import pandas as pd #data wrangling
import matplotlib.pyplot as plt #plotting package
#Next line helps with rendering plots
%matplotlib inline
import matplotlib as mpl #add'l plotting
functionality
mpl.rcParams['figure.dpi'] = 400 #high res figures
import graphviz #to visualize decision trees
```

2. Load the cleaned case study data:

```
df =
pd.read_csv('../Data/Chapter_1_cleaned_data.csv')
```

Note

The location of the cleaned data may differ depending on where you saved it.

3. Get a list of column names of the DataFrame:

```
features_response = df.columns.tolist()
```

4. Make a list of columns to remove that aren't features or the response variable:

```
items_to_remove = ['ID', 'SEX', 'PAY_2', 'PAY_3', \
                    'PAY_4', 'PAY_5', 'PAY_6', \
                    'EDUCATION_CAT', 'graduate
school', \
```

```
'high school', 'none',\n    'others', 'university']
```

5. Use a list comprehension to remove these column names from our list of features and the response variable:

```
features_response = [item for item in\n    features_response if item not in items_to_remove]\nfeatures_response
```

This should output the list of features and the response variable:

```
['LIMIT_BAL',\n    'EDUCATION',\n    'MARRIAGE',\n    'AGE',\n    'PAY_1',\n    'BILL_AMT1',\n    'BILL_AMT2',\n    'BILL_AMT3',\n    'BILL_AMT4',\n    'BILL_AMT5',\n    'BILL_AMT6',\n    'PAY_AMT1',\n    'PAY_AMT2',\n    'PAY_AMT3',\n    'PAY_AMT4',\n    'PAY_AMT5',\n    'PAY_AMT6',\n    'default payment next month']
```

Now the list of features is prepared. Next, we will make some imports from scikit-learn. We want to make a train/test split, which we are already familiar with. We also want to import the decision tree functionality.

6. Run this code to make imports from scikit-learn:

```
from sklearn.model_selection import train_test_split\nfrom sklearn import tree
```

The **tree** library of scikit-learn contains decision tree-related classes.

7. Split the data into training and testing sets using the same random seed that we have used throughout the book:

```
X_train, X_test, y_train, y_test = \
    train_test_split(df[features_response[:-1]].values,
                    df['default payment next
month'].values,
                    test_size=0.2, random_state=24)
```

Here, we use all but the last element of the list to get the names of the features, but not the response variable:

features_response[:-1]. We use this to select columns from the DataFrame, and then retrieve their values using the **.values** method. We also do something similar for the response variable, but specify the column name directly. In making the train/test split, we've used the same random seed as in previous work, as well as the same split size. This way, we can directly compare the work we will do in this chapter with previous results. Also, we continue to reserve the same "unseen test set" from the model development process.

Now we are ready to instantiate the decision tree class.

8. Instantiate the decision tree class by setting the **max_depth** parameter to 2:

```
dt = tree.DecisionTreeClassifier(max_depth=2)
```

We have used the **DecisionTreeClassifier** class because we have a classification problem. Since we specified **max_depth=2**, when we grow the decision tree using the case study data, the tree will grow to a depth of at most 2. Let's now train this model.

9. Use this code to fit the decision tree model and grow the tree:

```
dt.fit(X_train, y_train)
```

This should display the following output:

```
DecisionTreeClassifier(max_depth=2)
```

Now that we have fit this decision tree model, we can use the **graphviz** package to display a graphical representation of the tree.

10. Export the trained model in a format that can be read by the **graphviz** package using this code:


```
dot_data = tree.export_graphviz(dt,
                                out_file=None,
                                filled=True,
                                rounded=True,
                                feature_names=\
                                features_response[:-1]
                                ],
                                proportion=True,
                                class_names=[
                                    'Not defaulted',
                                    'Defaulted'])
```

Here, we've provided a number of options for the **.export_graphviz** method. First, we need to say which trained model we'd like to graph, which is **dt**. Next, we say we don't want an output file: **out_file=None**. Instead, we provide the **dot_data** variable to hold the output of this method. The rest of the options are set as follows:

filled=True: Each node will be filled with a color.

rounded=True: The nodes will appear with rounded edges as opposed to rectangles.

feature_names=features_response[:-1]: The names of the features from our list will be used as opposed to generic names such as **X[0]**.

proportion=True: The proportion of training samples in each node will be displayed (we'll discuss this more later).

class_names=['Not defaulted', 'Defaulted']: The name of the predicted class will be displayed for each node.

What is the output of this method?

If you examine the contents of **dot_data**, you will see that it is a long text string. The **graphviz** package can interpret this text string to create a visualization.

11. Use the **.Source** method of the **graphviz** package to create an image from **dot_data** and display it:

```
graph = graphviz.Source(dot_data)
```

graph

The output should look like this:

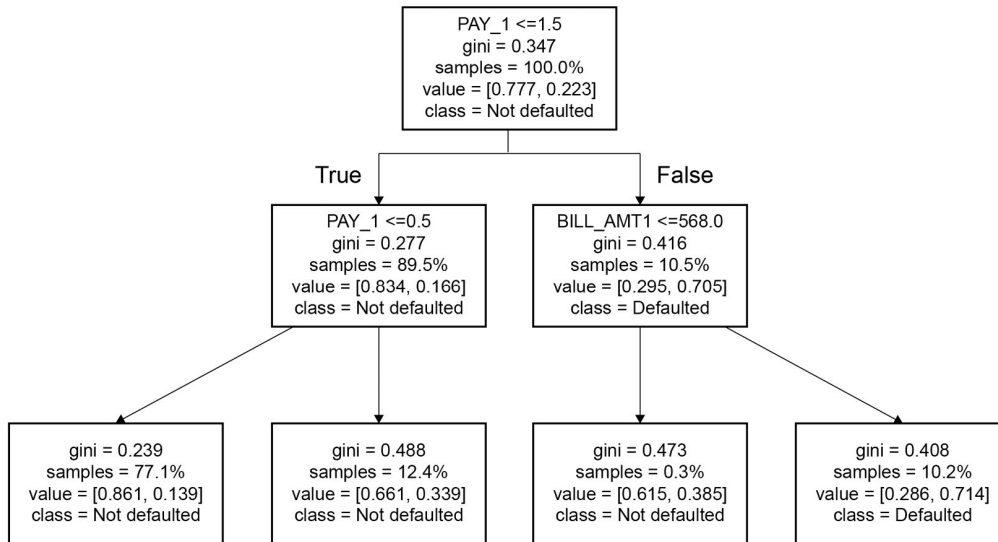


Figure 5.2: A decision tree plot from graphviz

The graphical representation of the decision tree in *Figure 5.2* should be rendered directly in your Jupyter notebook.

Note

Alternatively, you could save the output of `.export_graphviz` to disk by providing a file path to the `out_file` keyword argument. To turn this output file into an image file, for example, a `.png` file that you could use in a presentation, you could run this code at the command line, substituting in the filenames as appropriate: `$ dot -Tpng <exported_file_name> -o <image_file_name_you_want>.png`. For further details on the options relating to `.export_graphviz`, you should consult the scikit-learn documentation (https://scikit-learn.org/stable/modules/generated/sklearn.tree.export_graphviz.html).

The visualization in *Figure 5.2* contains a lot of information about how the decision tree was trained, and how it can be used to make predictions. We will discuss the training process in more detail later, but suffice to say that training a decision tree works by starting with all the training samples in the initial node at the top of the tree, and then splitting these into two groups based on a **threshold** in one of the features. The cut point is represented by the inequality `PAY_1 <= 1.5` in the first node.

All the samples where the value of the **PAY_1** feature is less than or equal to the cut point of **1.5** will be represented as **True** under this Boolean condition. As shown in *Figure 5.2*, these samples get sorted into the left side of the tree, following the arrow that says **True** next to it.

As you can see in the graph, each node that is split contains the splitting criteria on the first line of text. The next line relates to **gini**, which we will discuss shortly.

The following line contains information about the proportion of samples in each node. In the top node, we are starting with all the samples (**samples = 100.0%**). Following the first split, 89.5% of the samples get sorted into the node on the left, while the remaining 10.5% go into the node on the right. This information is shown directly in the visualization and reflects how the training data was used to create the tree. Let's confirm this by examining the training data.

12. To confirm the proportion of training samples where the **PAY_1** feature is less than or equal to **1.5**, first identify the index of this feature in the list of **features_response[: -1]** feature names:

```
features_response[: -1].index('PAY_1')
```

This code should output the following:

```
4
```

13. Now, observe the shape of the training data:

```
X_train.shape
```

This should give you the following output:

```
(21331, 17)
```

To confirm the fraction of samples following the first split of the decision tree, we need to know the proportion of samples, where the **PAY_1** feature meets the Boolean condition, that was used to make this split. To do this, we can use the index of the **PAY_1** feature in the training data, corresponding to the index in the list of feature names, and the number of samples in the training data, which is the number of rows we observed from **.shape**.

14. Use this code to confirm the proportion of samples after the first split of the decision tree:

```
(X_train[:,4] <= 1.5).sum()/X_train.shape[0]
```

The output should be as follows:

```
0.8946134733486475
```

By applying a logical condition to the column of the training data corresponding to the **PAY_1** feature, and then taking the sum of this, we calculated the number of samples meeting this condition. Then, by dividing by the total number of samples, we converted this to a proportion. We can see that the proportion we directly calculated from the training data is equal to the proportion displayed in the left node following the first split in *Figure 5.2*.

Following the first split, the samples contained in each of the two nodes on the first level are split again. As further splits are made beyond the first split, smaller and smaller proportions of the training data will be assigned to any given node in the subsequent levels of a branch, as can be seen in *Figure 5.2*.

Now we want to interpret the remaining lines of text in the nodes in *Figure 5.2*. The lines starting with **value** give the class fractions of the response variable for the samples contained in each node. For example, in the top node, we see **value = [0.777, 0.223]**. These are simply the class fractions for the overall training set, which you can confirm in the following step.

15. Calculate the class fraction in the training set with this code:

```
y_train.mean()
```

The output should be as follows:

```
0.223102526838873
```

This is equal to the second member of the pair of numbers following **value** in the top node; the first number is simply one minus this, in other words, the fraction of negative training samples. In each subsequent node, the class fractions of the samples that are contained in that node are displayed. The class fractions are also how the nodes are colored: those with a higher proportion of the negative class than the positive class are orange, with darker or-

ange signifying higher proportions, while those with a higher proportion of the positive class have a similar scheme using a blue color.

Finally, the line starting with **class** indicates how the decision tree would make predictions from a given node, if that node were a leaf node. Decision trees for classification make predictions by determining which leaf node a sample will be sorted into, given the values of the features, and then predicting the class of the majority of the training samples in that leaf node. This strategy means that the tree structure and the class proportions in the leaf nodes are pieces of information that are needed to make a prediction.

For example, if we've made no splits and we are forced to make a prediction knowing nothing but the class fractions for the overall training data, we will simply choose the majority class. Since most people don't default, the class on the top node is **Not defaulted**.

However, the class fractions in the nodes of deeper levels are different, leading to different predictions. How does scikit-learn decide the structure of the tree? We'll discuss the training process in the following section.

Importance of `max_depth`

Recall that the only hyperparameter we specified in this exercise was `max_depth`, that is, the maximum depth to which the decision tree can be grown during the model training process. It turns out that this is one of the most important hyperparameters. Without placing a limit on the depth, the tree will be grown until one of the other limitations, specified by other hyperparameters, takes effect. This can lead to very deep trees, with very many nodes. For example, consider how many leaf nodes there could be in a tree with a depth of 20. This would be 220 leaf nodes, which is over 1 million! Do we even have 1 million training samples to sort into all these nodes? In this case, we do not. It would clearly be impossible to grow such a tree, with every node before the final level being split, using this training data. However, if we

remove the `max_depth` limit and rerun the model training of this exercise, observe the effect:



Figure 5.3: A portion of the decision tree grown with no maximum depth

Here, we have shown a portion of the decision tree that is grown with the default options, which include `max_depth=None`, meaning no limit in terms of the depth of the tree. The entire tree is about twice as wide as the portion shown here. There are so many nodes that they only appear as very small orange or blue patches; the exact interpretation of each node is not important as we are just trying to illustrate how large trees can potentially be. It should be clear that without hyperparameters to govern the tree-growing process, extremely large and complex trees may result.

Training Decision Trees: Node Impurity

At this point, you should have an understanding of how a decision tree makes predictions using features, and the class fractions of training samples in the leaf nodes. Now, we will learn how decision trees are trained. The training process involves selecting features to split nodes on, and the thresholds at which to make splits, for example `PAY_1 <= 1.5` for the first split in the tree of the previous exercise.

Computationally, this means the samples in each node must be sorted on the values of each feature to consider a split for, and splits between each successive pair of sorted feature values are considered. All features may be considered, or only a subset as we will learn about shortly.

How Are the Splits Decided during the Training Process?

Given that the method of prediction is to take the majority class of a leaf node, it makes sense that we'd like to find leaf nodes that are primarily from one class or the other; choosing the majority class will be a more accurate prediction, the closer a node is to containing just one class. In the perfect case, the training data can be split so that every leaf node contains entirely positive or entirely negative samples. Then, we will have a high level of confidence that a new sample, once sorted into one of these nodes, will be either positive or negative. In practice, this rarely, if ever, happens. However, this illustrates the goal of training decision trees – that is, to make splits so that the next two nodes after the split have a higher **purity**, or, in other words, are closer to containing either only positive or only negative samples.

In practice, decision trees are actually trained using the inverse of purity, or **node impurity**. This is some measure of how far the node is from having 100% of the training samples belonging to one class and is analogous to the concept of a cost function, which signifies how far a given solution is from a theoretical perfect solution. The most intuitive concept of node impurity is the **misclassification rate**. Adopting a widely used notation (for example, <https://scikit-learn.org/stable/modules/tree.html>) for the proportion of samples in each node belonging to a certain class, we can define p_{mk} as the proportion of samples belonging to the k^{th} class in the m^{th} node. In a binary classification problem, there are only two classes: $k = 0$ and $k = 1$. For a given node m , the misclassification rate is simply the proportion of the less common class in that node, since all these samples will be misclassified when the majority class in that node is taken as the prediction.

Let's visualize the misclassification rate as a way to start thinking about how decision trees are trained. Programmatically, we consider possible class fractions, p_{m0} , between 0.01 and 0.99 of the negative class, $k = 0$, in a node, m , using NumPy's **linspace** function:

```
pm0 = np.linspace(0.01,0.99,99)
```

```
pm1 = 1 - pm0
```

Then, the fraction of the positive class for this node is one minus p_{m0} :

$$p_{m1} = 1 - p_{m0}$$

Figure 5.4: Equation for calculating the positive class fraction for node $m0$

Now, the misclassification rate for this node will be whatever the smaller class fraction is, between p_{m0} and p_{m1} . We can find the smaller of the corresponding elements between two arrays with the same shape in NumPy by using the **minimum** function:

```
misclassification_rate = np.minimum(pm0, pm1)
```

What does the misclassification rate look like plotted against the possible class fractions of the negative class?

We can plot this using the following code:

```
mpl.rcParams['figure.dpi'] = 400
plt.plot(pm0, misclassification_rate,
         label='Misclassification rate')
plt.xlabel('$p_{m0}$')
plt.legend()
```

You should obtain this graph:

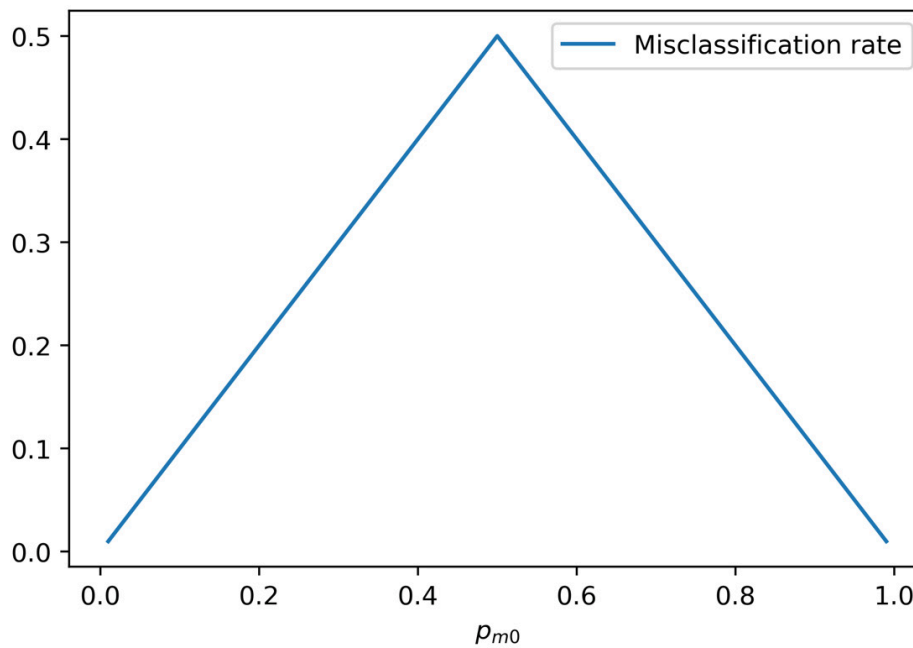


Figure 5.5: The misclassification rate for a node

Now, it's clear that the closer the class fraction of the negative class, p_{m0} , is to 0 or 1, the lower the misclassification rate will be. How is this information used when growing decision trees? Consider the process that might be followed.

Every time a node is split when growing a decision tree, two new nodes are created. Since the prediction from either of these new nodes is simply the majority class, an important goal will be to reduce the misclassification rate. Therefore, we will want to find a feature, from all the possible features, and a value of this feature at which to make a cut point, so that the misclassification rate in the two new nodes will be as low as possible when averaging over all the classes. This is very close to the actual process that is used to train decision trees.

Continuing for the moment with the idea of minimizing the misclassification rate, the decision tree training algorithm goes about node splitting by considering all the features, although the algorithm may possibly only consider a randomly selected subset if you set the **max_features** hyperparameter to anything less than the total number of features. We'll discuss possible reasons for doing this later. In either case, the algorithm then considers each possible threshold for every

candidate feature and chooses the one that results in the lowest impurity, calculated as the average impurity across the two possible new nodes, weighted by the number of samples in each node. The node splitting process is shown in *Figure 5.6*. This process is repeated until a stopping criterion of the tree, such as **max_depth**, is reached:

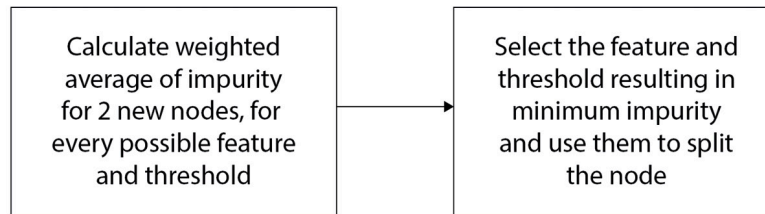


Figure 5.6: How to select a feature and threshold in order to split a node

While the misclassification rate is an intuitive measure of impurity, it happens that there are better measures that can be used to find splits during the model training process. The two options that are available in scikit-learn for the impurity calculation, which you can specify with the **criterion** keyword argument, are the **Gini impurity** and the **cross-entropy** options. Here, we will describe these options mathematically and show how they compare with the misclassification rate.

Gini impurity is calculated for a node m using the following formula:

$$Gini = \sum_k p_{mk}(1 - p_{mk})$$

Figure 5.7: Equation for calculating Gini impurity

Here, the summation is taken over all classes. In the case of a binary classification problem, there are only two classes, and we can write this programmatically as follows:

```
gini = (pm0*(1-pm0)) + (pm1*(1-pm1))
```

Cross-entropy is calculated using this formula:

$$\text{cross entropy} = - \sum_k p_{mk} \log(p_{mk})$$

Figure 5.8: Equation for calculating cross-entropy

Using this code, we can calculate the cross-entropy:

```
cross_ent = -1*((pm0*np.log(pm0)) + (pm1*np.log(pm1)))
```

In order to add Gini impurity and cross-entropy to our plot of misclassification rate and see how they compare, we just need to include the following lines of code after we plot the misclassification rate:

```
mpl.rcParams['figure.dpi'] = 400
plt.plot(pm0, misclassification_rate,\
         label='Misclassification rate')
plt.plot(pm0, gini, label='Gini impurity')
plt.plot(pm0, cross_ent, label='Cross entropy')
plt.xlabel('$p_{m0}$')
plt.legend()
```

The final plot should appear as follows:

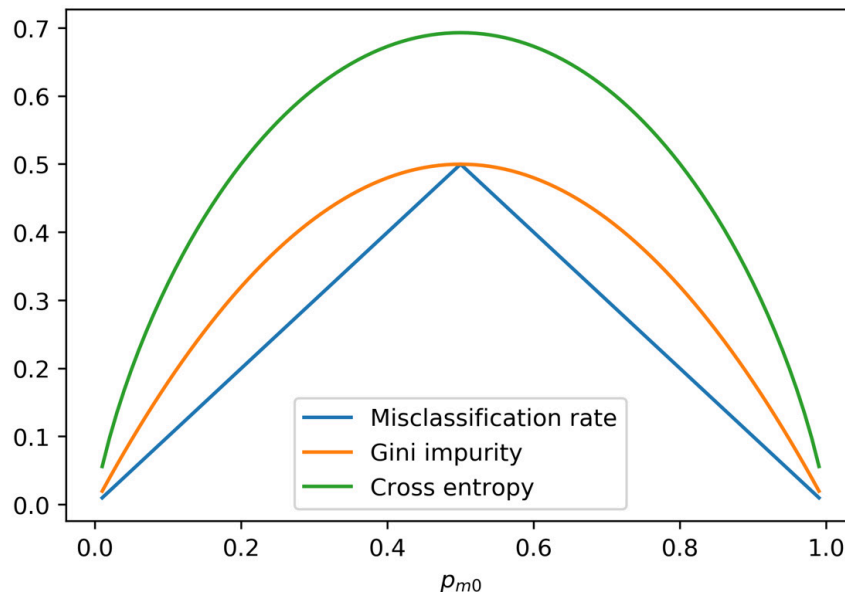


Figure 5.9: The misclassification rate, Gini impurity, and cross-entropy

Note

If you're reading the print version of this book, you can download and browse the color versions of some of the images in this chapter by visiting the following link:

<https://packt.link/mQ4Xn>

Like the misclassification rate, both the Gini impurity and cross-entropy are highest when the class fractions are equal at 0.5, and they decrease as the node becomes purer – in other words, when they contain a higher proportion of just one of the classes. However, the Gini impurity is somewhat steeper than the misclassification rate in certain regions of the class fraction, which enables it to more effectively find the best split. Cross-entropy looks even steeper. So, which one is better for your work? This is the kind of question that does not have a concrete answer across all datasets. You should consider both impurity metrics in a cross-validation search for hyperparameters in order to determine the appropriate one. Note that in scikit-learn, Gini impurity can be specified with the **criterion** argument using the **'gini'** string, while cross-entropy is just referred to as **'entropy'**.

Features Used for the First Splits: Connections to Univariate Feature Selection and Interactions

We can begin to get an impression of how important various features are to decision tree models, based on the small tree shown in *Figure 5.2*. Notice that **PAY_1** was the feature chosen for the first split. This means that it was the best feature in terms of decreasing node impurity on the node containing all of the training samples. Recall our experience with univariate feature selection in *Chapter 3, Details of Logistic Regression and Feature Exploration*, where **PAY_1** was the top-selected feature from the F-test. So, the appearance of this feature in the first split of the decision tree makes sense given our previous analysis.

In the second level of the tree, there is another split on **PAY_1**, as well as a split on **BILL_AMT_1**. **BILL_AMT_1** was not listed among the top features in univariate feature selection. However, it may be that there is an important interaction between **BILL_AMT_1** and **PAY_1**, which would not be found up by univariate methods. In particular, from the splits chosen by the decision tree, it seems that those accounts with both a value of 2 or greater for **PAY_1**, and a **BILL_AMT_1** of greater than 568, are especially at risk of default. This combined effect of **PAY_1** and **BILL_AMT_1** is an interaction and may also be why we were able to improve logistic regression performance by including interaction terms in the activity of the previous chapter.

Training Decision Trees: A Greedy Algorithm

There is no guarantee that a decision tree trained by the process described previously will be the best possible decision tree for finding leaf nodes with the lowest impurity. This is because the algorithm used to train decision trees is what is called a greedy algorithm. In this context, this means that at each opportunity to split a node, the algorithm is looking for the best possible split at that point in time, without any regard for the fact that the opportunities for later splits are being affected.

For example, consider the following hypothetical scenario: the best initial split for the training data of the case study involves **PAY_1**, as we've seen in *Figure 5.2*. But what if we instead split on **BILL_AMT_1**, and then make subsequent splits on **PAY_1** in the next level? Even though the initial split on **BILL_AMT_1** is not the best one available at first, it is possible that the end result will be better if the tree is grown this way. The algorithm has no way of finding solutions like this if they exist, since it only considers the best possible split at each node and not possible future splits.

The reason why we still use greedy tree-growing algorithms is that it takes substantially longer to consider all possible splits in a way that

enables the truly optimal tree to be found. Despite this shortcoming of the decision tree training process, there are methods that you can use to reduce the possible harmful effects of the greedy algorithm. Instead of searching for the best split at each node, the **splitter** keyword argument to the decision tree class can be set to **random** in order to choose a random feature to make a split on. However, the default is **best**, which searches all features for the best split. Another option, which we've already discussed, is to limit the number of features that will be searched at each splitting opportunity using the **max_features** keyword. Finally, you can also use ensembles of decision trees, such as random forests, which we will describe shortly. Note that all these options, in addition to possibly avoiding the ill-effects of the greedy algorithm, are also options for addressing the overfitting that decision trees are often criticized for.

Training Decision Trees: Different Stopping Criteria and Other Options

We have already reviewed using the **max_depth** parameter as a limit to how deep a tree will grow. However, there are several other options available in scikit-learn as well. These are mainly related to how many samples are present in a leaf node, or how much the impurity can be decreased by further splitting nodes. As discussed previously, you may be limited by the size of your dataset in terms of how deep you can grow a tree. And it may not make sense to grow trees deeper, especially if the splitting process is no longer finding nodes with substantially higher purity.

We summarize all of the keyword arguments that you can supply to the **DecisionTreeClassifier** class in scikit-learn here:

Parameter	Possible values	Notes
criterion	string, 'gini', or 'entropy'	The formula used to calculate node impurity.
splitter	string, 'best', or 'random'	Whether to search among all candidate features when making a split, or to choose one at random.
max_depth	int or None	Stopping criterion. <code>None</code> means there is no limit to the maximum depth for growing the tree, although the tree may be stopped for reasons specified in other hyperparameters. An integer means stop growing the tree after that many levels.
min_samples_split	int or float	Stopping criterion. If an integer, then a node must have at least this many samples in order to be split. If a float, it is the fraction of the total number of samples that must be in a node to split it.
min_samples_leaf	int or float	Stopping criterion. Similar to <code>min_samples_split</code> , but this refers to the number of samples that will be in the nodes following the split. Refers to a node at any depth.
min_weight_fraction_leaf	Float	Similar to <code>min_samples_leaf</code> , but uses the weight fraction of samples instead of the fraction of samples. Useful only if sample weighting has been specified with the <code>class_weight</code> parameter.
max_features	int, float, string: 'auto', 'sqrt', 'log2', or None	Strategy for how many features to consider when trying to split a node. If <code>None</code> , all the features are considered. If an <code>int</code> or <code>float</code> , that number or fraction of features are considered. 'auto' and 'sqrt' both use the square root of the number of features, and 'log2' uses the base 2 logarithm. If the number here is less than the total number of features, a random selection of features is taken. However, more than this number of features may be considered if none of the random selection satisfies other criteria such as <code>min_impurity_decrease</code> .
random_state	int or None	If an integer is supplied, this will seed the random number generator for repeatable results between runs of the same code.
max_leaf_nodes	int or None	Stopping criterion. The maximum number of leaf nodes if an integer is supplied, or no limit for <code>None</code> .
min_impurity_decrease	Float	Stopping criterion. If larger than 0, the required decrease in impurity to split a node. This means the tree only keeps growing if the nodes are getting purer.
min_impurity_split	Float	Stopping criterion, but being phased out. Use <code>min_impurity_decrease</code> instead.
class_weight	dict, list of dict, 'balanced', or None	If the response variable has imbalanced classes, 'balanced' will weight samples similar to logistic regression. Here, this would be used to calculate the weighted average of node impurity. A dictionary can be used to manually supply class weights. A list of dictionaries can be used for multi-output problems, which is when there are multiple response variables (beyond the scope of this book). <code>None</code> does no sample weighting.
ccp_alpha	Non-negative float	Related to a process called pruning, which means removing nodes after the tree is grown.

Figure 5.10: The complete list of options for the decision tree classifier in scikit-learn

Using Decision Trees: Advantages and Predicted Probabilities

While decision trees are simple in concept, they have several practical advantages.

No Need to Scale Features

Consider the reasons why we needed to scale features for logistic regression. One reason is that, for some of the solution algorithms based on gradient descent, it is necessary that the features are on the same scale in order to quickly find a minimum of the cost function. Another

is that when we are using L1 or L2 regularization to penalize coefficients, all the features must be on the same scale so that they are penalized equally. With decision trees, the node splitting algorithm considers each feature individually and, therefore, it doesn't matter whether the features are on the same scale.

Non-Linear Relationships and Interactions

Because each successive split in a decision tree is performed on a subset of the training samples resulting from the previous split(s), decision trees can describe complex non-linear relationships of a single feature, as well as interactions between features. Consider our discussion previously in the *Features Used for the First Splits: Connections to Univariate Feature Selection and Interactions* section. Also, as a hypothetical example with synthetic data, consider the following dataset for classification:

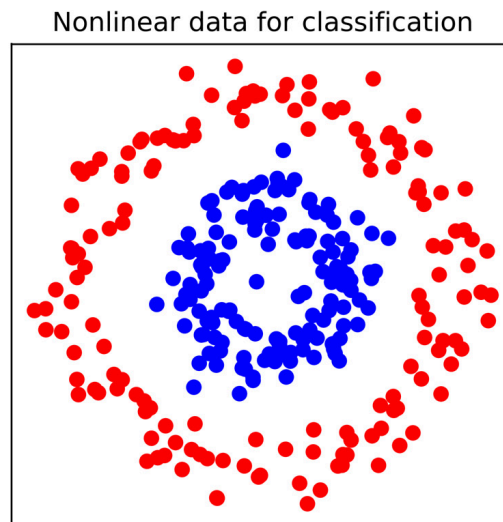


Figure 5.11: An example classification dataset, with the classes shown in red and blue (if reading in black and white, please refer to the GitHub repository for a color version of this figure; the blue dots are on the inside circle)

We know from *Chapter 3, Details of Logistic Regression and Feature Exploration*, that logistic regression has a linear decision boundary. So, how do you think logistic regression would cope with a dataset like that shown in *Figure 5.11*? Where would you draw a line to separate

the blue and red classes? It should be clear that without engineering additional features, a logistic regression is not likely to be a good classifier for this data. Now think about the set of "if, then" rules of a decision tree, which could be used with the features represented on the x and y axes of *Figure 5.11*. Do you think a decision tree will be effective with this data?

Here, we plot in the background the predicted probabilities of class membership using red and blue, for both of these models:

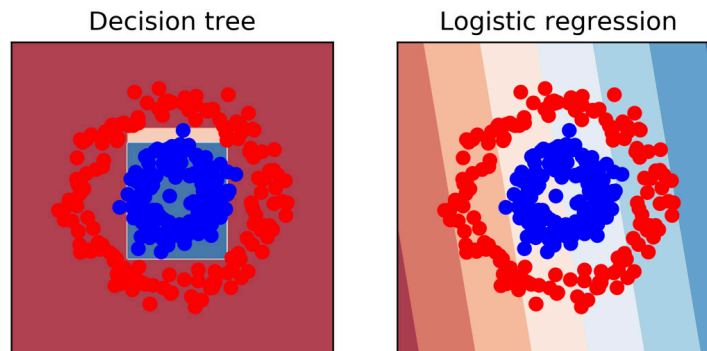


Figure 5.12: Decision tree and logistic regression predictions

In *Figure 5.12*, the predicted probabilities for both models are colored so that darker red corresponds to a higher predicted probability for the red class, and darker blue for the blue class. We can see that the decision tree can isolate the blue class in the middle of the circle of red points. This is because, by using thresholds for the x and y coordinates in the node-splitting process, a decision tree can mathematically model the fact that the location of the blue and red classes depends on both the x and y coordinates together (interactions), and that the likelihood of either class is not a linearly increasing or decreasing function of x or y (non-linearities). Consequently, the decision tree approach is able to get most classifications right.

Note

The code to generate Figures 5.11 and 5.12 can be found in the reference notebook: <https://packt.link/9W4WN>.

However, the logistic regression has a linear decision boundary, which will be the straight line between the lightest blue and red patches in the background. The logistic regression decision boundary goes right through the middle of the data and doesn't provide a useful classifier. This shows the power of decision trees "out of the box," without the need for engineering non-linear or interaction features.

Predicted Probabilities

We know that logistic regression produces probabilities as raw output. However, a decision tree makes predictions based on the majority class of the leaf nodes. So, where would predicted probabilities come from, like those shown in *Figure 5.12*? In fact, decision trees do offer the `.predict_proba` method in scikit-learn to calculate predicted probabilities. The probability is based on the proportion of the majority class in the leaf node used for a given prediction. If 75% of the samples in a leaf node belonged to the positive class, for example, the prediction for that node would be the positive class and the predicted probability will be 0.75. The predicted probabilities from decision trees are not considered to be as statistically rigorous as those from generalized linear models, but they are still commonly used to measure the performance of models by methods that depend on varying the threshold for classification, such as the ROC curve or the precision-recall curve.

Note

*We are focusing here on decision trees for classification because of the nature of the case study. However, decision trees can also be used for regression, making them a versatile method. The tree-growing process is similar for regression as it is for classification, except that instead of seeking to reduce node impurity, a regression tree seeks to minimize other metrics such as the **Mean Squared Error (MSE)** or **Mean Absolute Error (MAE)** of the predictions, where the prediction for a node may be the average or median of the samples in the node, respectively.*

A More Convenient Approach to Cross-Validation

In *Chapter 4, The Bias-Variance Trade-Off*, we gained a deep understanding of cross-validation by writing our own function to do it, using the **KFold** class to generate the training and testing indices. This was helpful to get a thorough understanding of how the process works. However, scikit-learn offers a convenient class that can do more of the heavy lifting for us: **GridSearchCV**. **GridSearchCV** can take as input a model that we want to find optimal hyperparameters for, such as a decision tree or a logistic regression, and a "grid" of hyperparameters that we want to perform cross-validation over. For example, in a logistic regression, we may want to get the average cross-validation score over all the folds for different values of the regularization parameter, **C**. With decision trees, we may want to explore different depths of trees.

You can also search multiple parameters at once, for example, if we wanted to try different depths of trees and different numbers of **max_features** to consider at each node split.

GridSearchCV does what is called an exhaustive grid search over all the possible combinations of parameters that we supply. This means that if we supplied five different values for each of the two hyperparameters, the cross-validation procedure would be run $5 \times 5 = 25$ times. If you are searching many values of many hyperparameters, the number of cross-validation runs can grow very quickly. In these cases, you may wish to use **RandomizedSearchCV**, which searches a random subset of hyperparameter combinations from the universe of all possibilities in the grid you supply.

GridSearchCV can speed up your work by streamlining the cross-validation process. You should be familiar with the concepts of cross-validation from the previous chapter, so we proceed directly to listing all the options available for **GridSearchCV**.

In the following exercise, we will get hands-on practice using **GridSearchCV** with the case study data, in order to search hyperparameters for a decision tree classifier. Here are the options for **GridSearchCV**:

Parameter	Possible values	Notes
Estimator	estimator object	This is a model object that you have instantiated from a model class. The hyperparameters will be updated as GridSearchCV does its work.
param_grid	dict or list of dict (dictionaries)	The dictionary has parameter names as keys and lists of parameters values as values. These are the hyperparameter values for which you want to search all possible combinations. To do multiple grid searches, supply a list of dictionaries.
Scoring	String, callable, list, tuple, or dict	This represents the model assessment metric(s) you want to use to measure training and testing performance across the folds, for example, 'roc_auc'.
n_jobs	int or None	The number of processing jobs to run in parallel. It may speed up cross-validation to run parallel jobs, but it is good to experiment to be sure.
pre_dispatch	int or string	The number of jobs or a formula for the number of jobs to dispatch. Relevant for parallel processing using n_jobs .
Cv	int, cross-validation generator, or iterable	If supplying an integer, this is the number of folds to use for cross-validation.
Refit	bool or string	After doing the cross-validation, the "best" hyperparameters according to the metric specified in scoring can be used directly with the fitted GridSearchCV object to make predictions. If refit=True , the model will be refit to all of the data (not just one of the folds) using the best hyperparameters. Use the string argument in case multiple metrics are specified, to indicate which one to use in order to choose the best hyperparameters.
Verbose	int	Controls how much output you will see from the cross-validation procedure.
error_score	'raise' or numeric	What to do if an error happens during model fitting.
return_train_score	bool	Whether or not to compute and return training scores on the folds. It is not required for selecting the best hyperparameters based on testing fold scores, and for some datasets and models, this can take substantially more time. However, it does give insights into possible overfitting.

Figure 5.13: The options for GridSearchCV

In the following exercise, we'll make use of the **standard error of the mean** to create error bars. We'll average the model performance metric across the testing folds, and the error bars will help us visualize how variable model performance is across the folds.

The standard error of the mean is also known as the standard deviation of the sampling distribution of the sample mean. That is a long name, but the concept isn't too complicated. The idea behind this is that the population of model performance metrics that we wish to make error bars for represents one possible way of sampling a theoretical, larger population of similar samples, for example if more data were available and we used it to have more testing folds. If we could

take repeated samples from the larger population, each of these sampling events would result in a slightly different mean (the sample mean). Constructing a distribution of these means (the sampling distribution of the sample mean) from repeated sampling events would allow us to know the variance of this sampling distribution, which would be useful as a measure of uncertainty in the sample mean. It

turns out this variance (let's call it $\sigma_{\bar{X}}^2$, where $\bar{X}$ indicates this is the variance of the sample mean) depends on the number of observations in our sample (n): it is inversely proportional to sample size, but also directly proportional to the variance of the larger, unobserved population (σ^2): $\sigma_{\bar{X}}^2 = \frac{\sigma^2}{n}$. If you're working with standard deviation of the

sample mean, simply take the square root of both sides: $\sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}}$. While we don't know the true value of σ^2 since we don't observe the theoretical population, we can estimate it with the variance of the population of testing folds that we do observe.

This is a key concept in statistics called the **Central Limit Theorem**.

Exercise 5.02: Finding Optimal Hyperparameters for a Decision Tree

In this exercise, we will use **GridSearchCV** to tune the hyperparameters for a decision tree model. You will learn about a convenient way of searching different hyperparameters with scikit-learn. Perform the following steps to complete the exercise:

Note

Before you begin this exercise, you need import the necessary packages and load the cleaned dataframe. You can refer to the following Jupyter notebook for the prerequisite steps: <https://packt.link/SKuoB>.

1. Import the **GridSearchCV** class with this code:

```
from sklearn.model_selection import GridSearchCV
```

The next step is to define the hyperparameters that we want to search using cross-validation. We will find the best maximum depth of tree, using the `max_depth` parameter. Deeper trees have more node splits, which partition the training set into smaller and smaller subspaces using the features. While we don't know the best maximum depth ahead of time, it is helpful to consider some limiting cases when considering the range of parameters to use for the grid search.

We know that one is the minimum depth, consisting of a tree with just one split. As for the largest depth, you can consider how many samples you have in your training data, or, more appropriately in this case, how many samples will be in the training fold for each split of the cross-validation. We will perform a 4-fold cross-validation like we did in the previous chapter. So, how many samples will be in each training fold, and how does this relate to the depth of the tree?

2. Find the number of samples in the training data using this code:

```
X_train.shape
```

The output should be as follows:

```
(21331, 17)
```

With 21,331 training samples and 4-fold cross-validation, there will be three-fourths of the samples, or about 16,000 samples, in each training fold.

What Does This Mean for How Deep We May Wish to Grow Our Tree?

A theoretical limitation is that we need at least one sample in each leaf. From our discussion regarding how the depth of the tree relates to the number of leaves, we know a tree that splits at every node before the last level, with n levels, has 2^n leaf nodes.

Therefore, a tree with L leaf nodes has a depth of approximately $\log_2(L)$. In the limiting case, if we grow the tree deep enough so that every leaf node has one training sample for a given fold, the depth will be $\log_2(16,000) \approx 14$. So, 14 is the theoretical limit to the depth of a tree that we could grow in this case.

Practically speaking, you will probably not want to grow a tree this deep, as the rules used to generate the decision tree will be very specific to the training data and the model is likely to be overfit. However, this gives you an idea of the range of values we may wish to consider for the **max_depth** hyperparameter. We will explore a range of depths from 1 up to 12.

3. Define a dictionary with the key being the hyperparameter name and the value being the list of values of this hyperparameter that we want to search in cross-validation:

```
params = {'max_depth':[1, 2, 4, 6, 8, 10, 12]}
```

In this case, we are only searching one hyperparameter. However, you could define a dictionary with multiple key-value pairs to search over multiple hyperparameters simultaneously.

4. If you are running all the exercises for this chapter in a single notebook, you can reuse the decision tree object, **dt**, from earlier. If not, you need to create a decision tree object for the hyperparameter search:

```
dt = tree.DecisionTreeClassifier()
```

Now we want to instantiate the **GridSearchCV** class.

5. Instantiate the **GridSearchCV** class using these options:

```
cv = GridSearchCV(dt, param_grid=params,  
                  scoring='roc_auc',  
                  n_jobs=None, refit=True, cv=4,  
                  verbose=1,  
                  pre_dispatch=None,  
                  error_score=np.nan,  
                  return_train_score=True)
```

Note here that we use the ROC AUC metric (**scoring='roc_auc'**), that we do 4-fold cross-validation (**cv=4**), and that we calculate training scores (**return_train_score=True**) to assess the bias-variance trade-off.

Once the cross-validation object is defined, we can simply use the **.fit** method on it as we would with a model object. This encapsu-

lates essentially all the functionality of the cross-validation loop we wrote in the previous chapter.

6. Perform 4-fold cross-validation to search for the optimal maximum depth using this code:

```
cv.fit(X_train, y_train)
```

The output should be as follows:

```
Fitting 4 folds for each of 7 candidates, totalling 28 fits
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 28 out of 28 | elapsed: 3.2s finished

GridSearchCV(cv=4, estimator=DecisionTreeClassifier(),
             param_grid={'max_depth': [1, 2, 4, 6, 8, 10, 12]},
             pre_dispatch=None, return_train_score=True, scoring='roc_auc',
             verbose=1)
```

Figure 5.14: The cross-validation fitting output

All the options that we specified are printed as output. Additionally, there is some output information regarding how many cross-validation fits were performed. We had 4 folds and 7 hyperparameters, meaning $4 \times 7 = 28$ fits are performed. The amount of time this took is also displayed. You can control how much output you get from this procedure with the **verbose** keyword argument; larger numbers mean more output.

Now it's time to examine the results of the cross-validation procedure. Among the methods that are available on the fitted

GridSearchCV object is **.cv_results_**. This is a dictionary containing the names of results as keys and the results themselves as values. For example, the **mean_test_score** key holds the average test score across the folds for each of the seven hyperparameters. You could directly examine this output by running **cv.cv_results_** in a code cell. However, this is not easy to read. Dictionaries with this kind of structure can be used immediately in the creation of a pandas DataFrame, which makes looking at the results a little easier.

7. Run the following code to create and examine a pandas DataFrame of cross-validation results:

```
cv_results_df = pd.DataFrame(cv.cv_results_)
cv_results_df
```

The output should look like this:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_max_depth	params	split0_test_score
0	0.023161	0.002917	0.002712	0.000827	1	{'max_depth': 1}	0.639514
1	0.040478	0.005298	0.003616	0.001432	2	{'max_depth': 2}	0.695134
2	0.062975	0.001703	0.002196	0.000017	4	{'max_depth': 4}	0.732720
3	0.094926	0.005329	0.002393	0.000150	6	{'max_depth': 6}	0.743836
4	0.123850	0.008124	0.003107	0.000768	8	{'max_depth': 8}	0.727948
5	0.142454	0.001005	0.002620	0.000221	10	{'max_depth': 10}	0.709049
6	0.178841	0.016180	0.002716	0.000238	12	{'max_depth': 12}	0.675597

Figure 5.15: First several columns of the cross-validation results DataFrame

The DataFrame has one row for each combination of hyperparameters in the grid. Since we are only searching one hyperparameter here, there is one row for each of the seven values that we searched for. You can see a lot of output for each row, such as the mean and standard deviation of the time in seconds that each of the four folds took for both training (fitting) and testing (scoring). The hyperparameter values that were searched are also shown. In *Figure 5.16*, we can see the ROC AUC score for the testing data of the first fold (index 0). What are the rest of the columns in the results DataFrame?

8. View the names of the remaining columns in the results DataFrame using this code:

```
cv_results_df.columns
```

The output should be as follows:

```
Index(['mean_fit_time', 'std_fit_time', \
      'mean_score_time', 'std_score_time', \
      'param_max_depth', 'params', \
      'split0_test_score', 'split1_test_score', \
      'split2_test_score', 'split3_test_score', \
      'mean_test_score', 'std_test_score', \
      'rank_test_score', 'split0_train_score', \
      'split1_train_score', 'split2_train_score', \
      'split3_train_score', 'mean_train_score', \
      'std_train_score'], \
      dtype='object')
```

The columns in the cross-validation results DataFrame include the testing scores for each fold, their average and standard deviation, and the same information for the training scores.

Generally speaking, the "best" combination of hyperparameters is that with the highest average testing score. This is an estimation of how well the model, fitted using these hyperparameters, could perform when scored on new data. Let's make a plot showing how the average testing score varies with the **max_depth** hyperparameter.

We will also show the average training scores on the same plot, to see how bias and variance change as we allow deeper and more complex trees to be grown during model fitting.

We include the standard errors of the 4-fold training and testing scores as error bars, using the Matplotlib **errorbar** function. This gives you an indication of how variable the scores are across the folds.

9. Execute the following code to create an error bar plot of training and testing scores for each value of **max_depth** that was examined in cross-validation:

```
ax = plt.axes()
ax.errorbar(cv_results_df['param_max_depth'],
            cv_results_df['mean_train_score'],
            yerr=cv_results_df['std_train_score']/np.
sqrt(4),
            label='Mean  $\pm$  1 SE training scores')
ax.errorbar(cv_results_df['param_max_depth'],
            cv_results_df['mean_test_score'],
            yerr=cv_results_df['std_test_score']/np.s
qrt(4),
            label='Mean  $\pm$  1 SE testing scores')
ax.legend()
plt.xlabel('max_depth')
plt.ylabel('ROC AUC')
```

The plot should appear as follows:

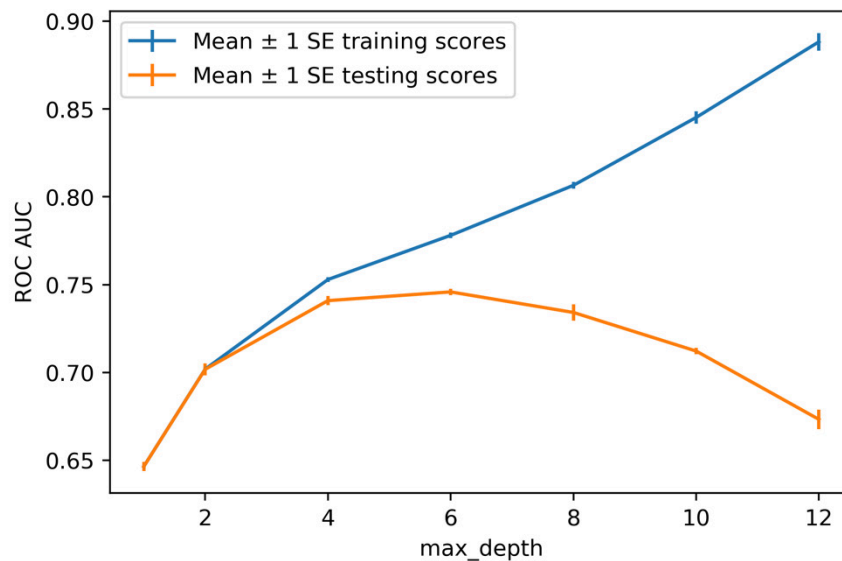


Figure 5.16: An error bar plot of training and testing scores across the four folds

Note that standard errors are calculated as the standard deviation divided by the square root of the number of folds. The standard errors of the training and testing scores are shown as vertical lines at each value of `max_depth` that was tried; the distance above and below the average score is 1 standard error. Whenever making error bar plots, it's best to ensure that the units of the error measurement are the same as the units of the y axis. In this case they are, since standard error has the same units as the underlying data, as opposed to variance, for example, which has squared units.

The error bars indicate how variable the scores are across folds. If there were a large amount of variation across the folds, it would indicate that the nature of the data across the folds was different in a way that affected the ability of our model to describe it. This could be concerning because it would indicate that we may not have enough data to train a model that would reliably perform on new data. However, in our case here, there is not much variability between the folds, so this is not an issue.

What about the general trends of the training and testing scores across the different values of `max_depth`? We can see that as we grow deeper and deeper trees, the model fits the training data better and better. As noted previously, if we grew trees deep enough so that each leaf node had just one training sample, we would create a model that is very specific to the training data. In fact, it would fit the training data perfectly. We could say that such a model had extremely high **variance**.

But this performance on the training set does not necessarily translate over to the testing set. In *Figure 5.16* it's apparent that increasing `max_depth` only increases testing scores up to a point, after which deeper trees in fact have lower testing performance. This is another example of how we can leverage the **bias-variance trade-off** to create a better predictive model – similar to how we used a regularized logistic regression. Shallower trees have more **bias**, since they are not fitting the training data as well. But this is fine because if we accept some bias, we will have better performance on the testing data, which is the metric we ultimately care about.

In this case, we would select `max_depth` = 6. You could also perform a more thorough search by trying every integer between 2 and 12, instead of going by 2s, as we've done here. In general, it is a good idea to perform as thorough a search of parameter space as you can, up to the limits of the computational time that you have. In this case, it would lead to the same result.

Comparison between Models

At this point, we've calculated a 4-fold cross-validation of several different machine learning models on the case study data. So, how are we doing? What's our best so far? In the last chapter, we got an average testing ROC AUC of 0.718 with logistic regression, and 0.740 by engineering interaction features in a logistic regression. Here, with a decision tree, we can achieve 0.745. So, we are making gains in model

performance. Now, let's, explore another type of model, based on decision trees, to see whether we can push performance even higher.

Random Forests: Ensembles of Decision Trees

As we saw in the previous exercise, decision trees are prone to overfitting. This is one of the principal criticisms of their usage, despite the fact that they are highly interpretable. We were able to limit this overfitting, to an extent, however, by limiting the maximum depth to which the tree could be grown.

Building on the concepts of decision trees, machine learning researchers have leveraged multiple trees as the basis for more complex procedures, resulting in some of the most powerful and widely used predictive models. In this chapter, we will focus on random forests of decision trees. Random forests are examples of what are called ensemble models, because they are formed by combining other, simpler models. By combining the predictions of many models, it is possible to improve upon the deficiencies of any given one of them. This is sometimes called combining many weak learners to make a strong learner.

Once you understand decision trees, the concept behind random forests is fairly simple. That is because random forests are just ensembles of many decision trees; all the models in this kind of ensemble have the same mathematical form. So, how many decision tree models will be included in a random forest? This is one of the hyperparameters, **n_estimators**, that needs to be specified when building a random forest model. Generally speaking, the more trees, the better. As the number of trees increases, the variance of the overall ensemble will decrease. This should result in the random forest model having better generalization to new data, which will be reflected in increased testing scores. However, there will be a point of diminishing returns after

which increasing the number of trees does not result in a substantial improvement in model performance.

So, how do random forests reduce the high variance (overfitting) issue that affects decision trees? The answer to this question lies in what is different about the different trees in the forest. There are two main ways in which the trees are different, one of which we are already familiar with:

- The number of features considered at each split
- The training samples used to grow different trees

The Number of Features Considered at Each Split

We are already familiar with this option from the **DecisionTreeClassifier** class: **max_features**. In our previous usage of this class, we left **max_features** at its default value of **None**, which meant that all features were considered at each split. By using all the features to fit the training data, overfitting is possible. By limiting the number of features considered at each split, some of the decision trees in a random forest will potentially find better splits. This is because, although they are still greedily searching for the best split, they are doing it with a limited selection of features. This may make certain splits possible later in the tree that may not have been found if all features were being searched at each split.

There is a **max_features** option in the **RandomForestClassifier** class in scikit-learn just as there is for the **DecisionTreeClassifier** class and the options are similar. However, for the random forest, the default setting is **'auto'**, which means the algorithm will only search a random selection of the square root of the number of possible features at each split, for example, a random selection of $\sqrt{9} = 3$ features from a total of 9 possible features. Because each tree in the forest will likely choose different random selections of features to split as the trees are being grown, the trees in the forest will not be the same.

The Samples Used to Grow Different Trees

The other way that the trees in a random forest differ from each other is that they are usually grown with different training samples. To do this, a statistical procedure known as bootstrapping is used, which means generating new synthetic datasets from the original data. The synthetic datasets are created by randomly selecting samples from the original dataset using replacement. Here, "replacement" means that if we select a certain sample, we will continue to consider it for selection, that is, it is "replaced" in the original dataset after we've sampled it. The number of samples in the synthetic datasets are the same as those in the original dataset, but some samples may be repeated because of replacement, while others may not be present at all.

The procedure of using random sampling to create synthetic datasets, and training models on them separately, is called bagging, which is short for bootstrapped aggregation. Bagging can, in fact, be used with any machine learning model, not just decision trees, and scikit-learn offers functionality to do this for both classification (**BaggingClassifier**) and regression (**BaggingRegressor**) problems. In the case of random forest, bagging is turned on by default and the **bootstrap** option is set to **True**. But if you want all the trees in the forest to be grown using all of the training data, you can set this option to **False**.

Now you should have a good understanding of what a random forest is. As you can see, if you are already familiar with decision trees, understanding random forests does not involve much additional knowledge. A reflection of this fact is that the hyperparameters available for the **RandomForestClassifier** class in scikit-learn are mostly the same as those for the **DecisionTreeClassifier** class.

In addition to **n_estimators** and **bootstrap**, which we discussed previously, there are only two new options beyond what's available for decision trees:

- **oob_score**, a **bool**: This option controls whether or not to calculate an **Out Of Bag (OOB)** score for each tree. This can be thought of as a testing score, where the samples that were not selected by the bagging procedure to grow a given tree are used to assess the model performance of that tree. Here, use **True** to calculate the OOB score or **False** (the default) not to.
- **warm_start**, a **bool**: This is **False** by default – if you set this to **True**, then reusing the same random forest model object will cause additional trees to be added to the already generated forest.
- **max_samples**, an **int** or **float**: Controls how many samples are used to train each tree in the forest, when using the bootstrapping procedure. The default is to use the same number as the original dataset.

Other Kinds of Ensemble Models

Random forest, as we now know, is an example of a bagging ensemble. Another kind of ensemble is a **boosting** ensemble. The general idea of boosting is to use successive new models of the same type and to train them on the errors of previous models. This way, successive models learn where earlier models didn't do well and correct these errors. Boosting has enjoyed successful application with decision trees and is available in scikit-learn and another popular Python package called XGBoost. We will discuss boosting in the next chapter.

Stacking ensembles are a somewhat more advanced kind of ensemble, where the different models (estimators) within the ensemble do not need to be of the same type as they do in bagging and boosting. For example, you could build a stacking ensemble with a random forest and a logistic regression. The predictions of the different members of the ensemble are combined for a final prediction using yet another model (the **stacker**), which considers the predictions of the **stacked** models as features.

Random Forest: Predictions and Interpretability

Since a random forest is just a collection of decision trees, somehow the predictions of all those trees must be combined to create the prediction of the random forest.

After model training, classification trees will take an input sample and produce a predicted class, for example, whether or not a credit account in our case study problem will default. One intuitive approach to combining the predictions of these trees into the ultimate prediction of the forest is to take a majority vote. That is, whatever the most common prediction of all the trees is becomes the prediction of the forest, for a given sample. This was the approach taken in the publication first describing random forests (<https://scikit-learn.org/stable/modules/ensemble.html#forest>). However, scikit-learn uses a somewhat different approach: adding up the predicted probabilities for each class and then choosing the one with the highest probability sum. This captures more information from each tree than just the predicted class.

Interpretability of Random Forests

One of the main advantages of decision trees is that it is straightforward to see how any individual prediction is made. You can trace the decision path for any sample through the series of "if, then" rules used to make a prediction and know exactly how it came to have that prediction. By contrast, imagine that you have a random forest consisting of 1,000 trees. This would mean there are 1,000 sets of rules like this, which are much harder to communicate to human beings than one set of rules!

That being said, there are various methods that can be used to understand how random forests make predictions. A simple way to interpret how a random forest works, and which is available in scikit-learn, is to observe the **feature importances**. Feature importances of a random forest are a measure of how useful each of the features was when growing the trees in the forest. This usefulness is measured by a com-

bination of the fraction of training samples that were split using each feature, and the decrease in node impurity that resulted.

Because of the feature importance calculation, which can be used to rank features by how impactful they are within the random forest model, random forests can also be used for feature selection.

Exercise 5.03: Fitting a Random Forest

In this exercise, we will extend our efforts with decision trees by using the random forest model with cross-validation on the training data from the case study. We will observe the effect of increasing the number of trees in the forest and examine the feature importance that can be calculated using a random forest model. Perform the following steps to complete the exercise:

Note

The Jupyter notebook for this exercise can be found at <https://packt.link/VSz2T>. This notebook contains the prerequisite steps of importing the necessary libraries and loading the cleaned dataframe. Please execute these steps before you begin this exercise.

1. Import the random forest classifier model class as follows:

```
from sklearn.ensemble import RandomForestClassifier
```

2. Instantiate the class using these options:

```
rf = RandomForestClassifier(n_estimators=10,\n                           criterion='gini',\n                           max_depth=3,\n                           min_samples_split=2,\n                           min_samples_leaf=1,\n                           min_weight_fraction_leaf=\n\n0.0,\n                           max_features='auto',\n                           max_leaf_nodes=None,\n
```

```

min_impurity_decrease=0.0
,\
min_impurity_split=None,\
bootstrap=True,\
oob_score=False,\
n_jobs=None,
random_state=4,\
verbose=0,\
warm_start=False,\
class_weight=None)

```

For this exercise, we'll use mainly the default options. However, note that we will set **max_depth** = 3. Here, we are only going to explore the effect of using different numbers of trees, which we will illustrate with relatively shallow trees for the sake of shorter run-times. To find the best model performance, we'd typically try more trees and deeper depths of trees.

We also set **random_state** for consistent results across runs.

3. Create a parameter grid for this exercise in order to search the numbers of trees, ranging from 10 to 100 by 10s:

```

rf_params_ex =
{'n_estimators':list(range(10,110,10))}

```

We use Python's **range()** function to create an iterator for the integer values we want, and then convert them to a list using **list()**.

4. Instantiate a grid search cross-validation object for the random forest model using the parameter grid from the previous step.

Otherwise, you can use the same options that were used for the cross-validation of the decision tree:

```

cv_rf_ex = GridSearchCV(rf, param_grid=rf_params_ex,
                        scoring='roc_auc',
n_jobs=None,
                        refit=True, cv=4, verbose=1,
                        pre_dispatch=None,
error_score=np.nan,
                        return_train_score=True)

```

5. Fit the cross-validation object as follows:

```
cv_rf_ex.fit(X_train, y_train)
```

The fitting procedure should output the following:

Fitting 4 folds for each of 10 candidates, totalling 40 fits

```
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 40 out of 40 | elapsed: 25.2s finished
```

```
GridSearchCV(cv=4,
             estimator=RandomForestClassifier(max_depth=3, n_estimators=10,
                                             random_state=4),
             param_grid={'n_estimators': [10, 20, 30, 40, 50, 60, 70, 80, 90,
                                           100]},
             pre_dispatch=None, return_train_score=True, scoring='roc_auc',
             verbose=1)
```

Figure 5.17: The output from the cross-validation of the random forest across different numbers of trees

You may have noticed that, although we are only cross-validating over 10 hyperparameter values, comparable to the 7 values that we examined for the decision tree in the previous exercise, this cross-validation took noticeably longer. Consider how many trees we are growing in this case. For the last hyperparameter, **n_estimators = 100**, we have grown a total of 400 trees across all the cross-validation splits.

How long has model fitting taken across the various numbers of trees that we just tried? What gains in terms of cross-validation testing performance have we made by using more trees? These are good things to examine using plots. First, we'll pull the cross-validation results out into a pandas DataFrame, as we've done before.

6. Put the cross-validation results into a pandas DataFrame:

```
cv_rf_ex_results_df =
pd.DataFrame(cv_rf_ex.cv_results_)
```

You can examine the whole DataFrame in the accompanying Jupyter notebook. Here, we move directly to creating plots of the quantities of interest. We'll make a line plot, with symbols, of the mean fit time across the folds for each hyperparameter, contained in the **mean_fit_time** column, as well as an error bar plot of testing scores, which we've already done for decision trees. Both plots will be against the number of trees on the x axis.

7. Create two subplots of the mean training time and mean testing scores with standard error:

```

fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(6,
3))
axs[0].plot(cv_rf_ex_results_df['param_n_estimators']
,
cv_rf_ex_results_df['mean_fit_time'],
'-o')
axs[0].set_xlabel('Number of trees')
axs[0].set_ylabel('Mean fit time (seconds)')
axs[1].errorbar(cv_rf_ex_results_df['param_n_estimators'],
cv_rf_ex_results_df['mean_test_score'],
yerr=cv_rf_ex_results_df['std_test_score']/np.sqrt(4))
axs[1].set_xlabel('Number of trees')
axs[1].set_ylabel('Mean testing ROC AUC  $\pm$  1 SE ')
plt.tight_layout()

```

Here, we've used **plt.subplots** to create two axes at once, within a figure, in a one-row-by-two-column configuration. We then access the axes objects by indexing the array of **axs** axes returned from this operation in order to create plots.

The output should look similar to this plot:

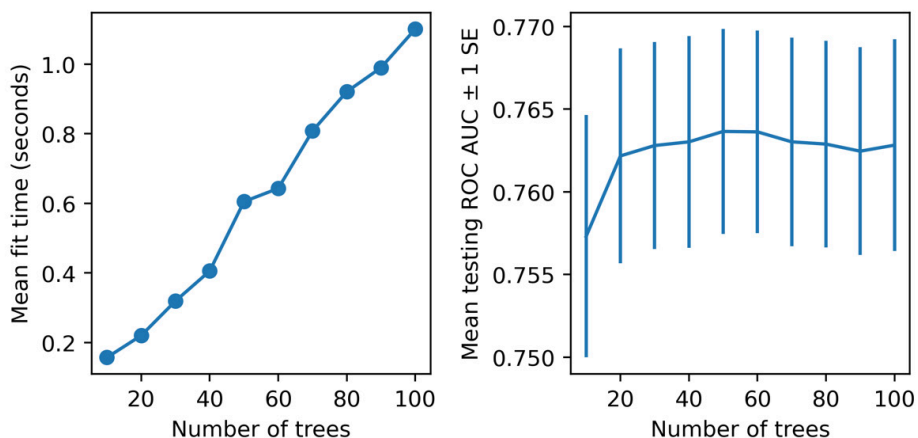


Figure 5.18: The mean fitting time and testing scores for different numbers of trees in the forest

Note

Your results may differ due to the differences in the platform or if you set a different random seed.

There are several things to note regarding these visualizations.

First of all, we can see that by using a random forest, we have increased model performance on the cross-validation testing folds above that of any of our previous efforts. While we haven't made an attempt to tune the random forest hyperparameters to achieve the best model performance we can, this is a promising result and indicates that a random forest will be a valuable addition to our modeling efforts.

However, along with these higher model testing scores, notice that there is also more variability between the folds than what we saw with the decision tree; this variability is visible as larger standard errors in model testing scores across the folds. While this indicates that there is a wider range in model performance that might be expected from using this model, you are encouraged to examine the model testing scores of the folds directly in the pandas DataFrame in the Jupyter notebook. You should see that even the lowest score from an individual fold is still higher than the average testing score from the decision tree, indicating that it will be better to use a random forest.

So, what about the other questions that we set out to explore with this visualization? We are interested in seeing how long it takes to fit random forest models with various numbers of trees, and what the gains in model performance are from using more trees. The subplot on the left of *Figure 5.18* shows that there is a fairly linear increase in training time as more trees are added to the forest. This is probably to be expected; we are simply adding to the amount of computation to be done in the training procedure by adding more trees.

But is this additional computational time worth it in terms of increased model performance? The subplot on the right of *Figure 5.18* shows that beyond about 20 trees, it's not clear that adding more trees reliably improves testing performance. While the model with 50 trees has the highest score, the fact that adding more trees

actually decreases the testing score somewhat indicates that the gain in ROC AUC for 50 trees may just be due to randomness, as adding more trees theoretically should increase model performance. Based on this reasoning, if we were limited to `max_depth = 3`, we may choose a forest of 20 or perhaps 50 trees and proceed. However, we will explore the parameter space more fully in the activity at the end of this chapter.

Finally, note that we have not shown the training ROC AUC metrics here. If you were to plot these or look them up in the results DataFrame, you'd see that the training scores are higher than the testing scores, indicating that some amount of overfitting is happening. While this may be the case, it's still true that the cross-validation testing scores for this random forest model are higher than those that we've observed for any other model. Based on this result, we would likely choose the random forest model at this point. For a few additional insights into what we can access using our fitted cross-validation object, let's take a look at the best hyperparameters and feature importance.

8. Use this code to see the best hyperparameters from cross-validation:

```
cv_rf_ex.best_params_
```

This should be the output:

```
{'n_estimators': 50}
```

Here, best just means the hyperparameters that resulted in the highest average model testing score.

9. Run this code to create a DataFrame of the feature names and importances, and then show a horizontal bar plot sorted by importance:

```
feat_imp_df = pd.DataFrame({  
    'Importance':cv_rf_ex.best_estimator_.feature_importances_  
    },  
    index=features_response[:-1])
```

```
feat_imp_df.sort_values('Importance',  
ascending=True).plot.barh()
```

The plot should look like this:

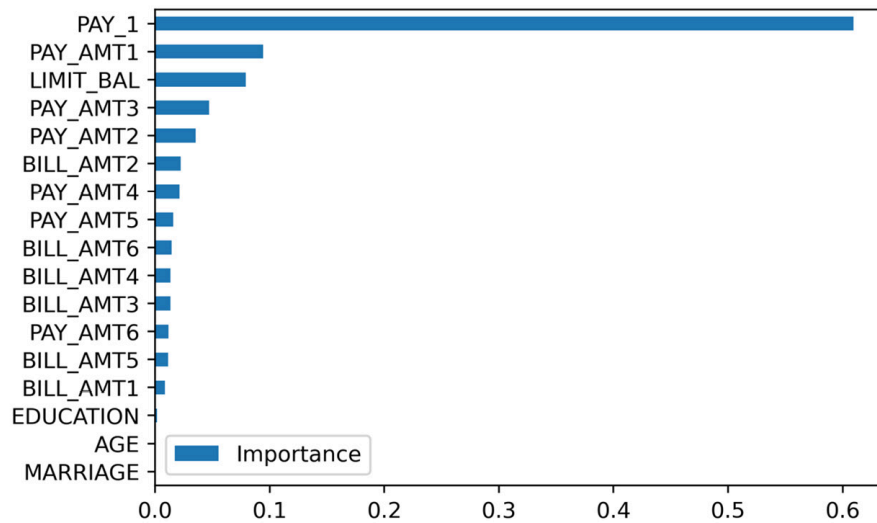


Figure 5.19: Feature importance from a random forest

In this code, we've created a dictionary with feature importances and used this along with the feature names as an index to create a DataFrame. The feature importances came from the **best_estimator_** method of the fitted cross-validation object, so it refers to the model with the highest average testing score (in other words, the model with 50 trees). This is a way to access the random forest model object, which was trained on all the training data, using the best hyperparameters found by the cross-validation grid search. **feature_importances_** is a method that can be used on fitted random forest models.

After accessing all these attributes, we plot them on a horizontal bar chart, which is a convenient way to look at feature importances. Notice that the top five most important features from the random forest are the same as the top five chosen by an ANOVA F-test in *Chapter 3, Details of Logistic Regression and Feature Exploration*, although they are in a somewhat different order. This is good confirmation between the different methods.

Checkerboard Graph

Before moving on to the activity, we illustrate a visualization technique in Matplotlib. Plotting a two-dimensional grid with colored squares or other shapes on it can be useful when you want to show three dimensions of data. Here, color illustrates the third dimension. For example, you may want to visualize model testing scores over a grid of two hyperparameters, as we'll do in *Activity 5.01, Cross-Validation Grid Search with Random Forest*.

The first step in the process is to create grids of x and y coordinates. The NumPy `meshgrid` function can be used to do this. This function takes one-dimensional arrays of x and y coordinates and creates the mesh grid with all the possible pairs from both. The points in the mesh grid will be the corners of each square on the checkerboard plot. Here is how the code looks for a 4 x 4 grid of colored patches. Since we are specifying the corners, we require a 5 x 5 grid of points. We also show the arrays of the x and y coordinates:

```
xx_example, yy_example = np.meshgrid(range(5),
range(5))
print(xx_example)
print(yy_example)
```

The output is as follows:

```
[[0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]]
[[0 0 0 0 0]
 [1 1 1 1 1]
 [2 2 2 2 2]
 [3 3 3 3 3]
 [4 4 4 4 4]]
```

The grid of data to plot on this mesh should have a 4 x 4 shape. We make a one-dimensional array of integers between 1 and 16, and re-

shape it to a two-dimensional, 4 x 4 grid:

```
z_example = np.arange(1,17).reshape(4,4)
z_example
```

This outputs the following:

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

We can plot the **z_example** data on the **xx_example**, **yy_example** mesh grid with the following code. Notice that we use **pcolormesh** to make the plot with the **jet** colormap, which gives a rainbow color scale. We add a **colorbar**, which needs to be passed the **pcolor_ex** object returned by **pcolormesh** as an argument, so the interpretation of the color scale is clear:

```
ax = plt.axes()
pcolor_ex = ax.pcolormesh(xx_example, yy_example,
                          z_example,
                          cmap=plt.cm.jet)
plt.colorbar(pcolor_ex, label='Color scale')
ax.set_xlabel('X coordinate')
ax.set_ylabel('Y coordinate')
```

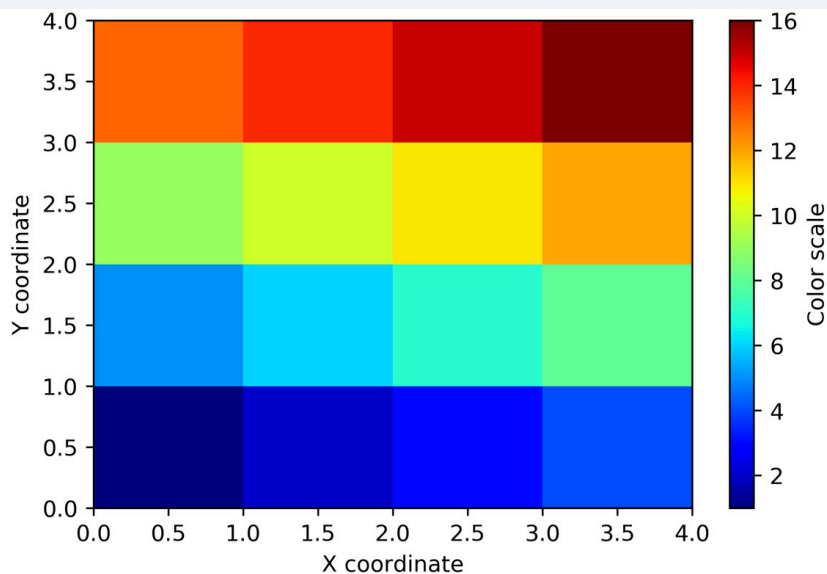


Figure 5.20: A pcolormesh plot of consecutive integers

Activity 5.01: Cross-Validation Grid Search with Random Forest

In this activity, you will conduct a grid search over the number of trees in the forest (**n_estimators**) and the maximum depth of a tree (**max_depth**) for a random forest model on the case study data. You will then create a visualization showing the average testing score for the grid of hyperparameters that you searched over. Perform the following steps to complete the activity:

1. Create a dictionary representing the grid for the **max_depth** and **n_estimators** hyperparameters that will be searched. Include depths of 3, 6, 9, and 12, and 10, 50, 100, and 200 trees. Leave the other hyperparameters at their defaults.
2. Instantiate a **GridSearchCV** object using the same options that we have had previously in this chapter, but with the dictionary of hyperparameters created in step 1 here. Set **verbose=2** to see the output for each fit performed. You can reuse the same random forest model object, **rf**, that we have been using or create a new one.
3. Fit the **GridSearchCV** object on the training data.
4. Put the results of the grid search in a pandas DataFrame.
5. Create a **pcolormesh** visualization of the mean testing score for each combination of hyperparameters. You should obtain a visualization similar to the following:

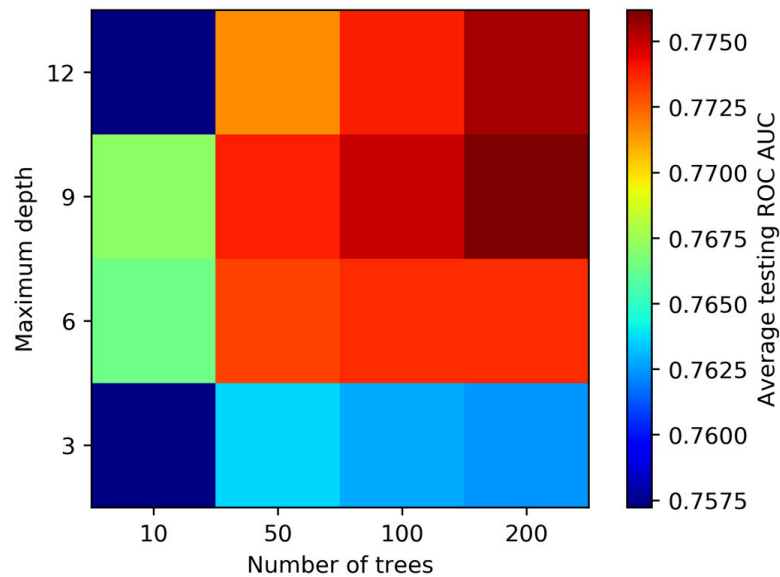


Figure 5.21: Results of cross-validation of a random forest over a grid with two hyperparameters

6. Conclude which set of hyperparameters to use.

Note

The Jupyter notebook containing the Python code for this activity can be found at <https://packt.link/D0OBc>. Detailed step-wise solution to this activity can be found via [this link](#).

Summary

In this chapter, we've learned how to use decision trees and the ensemble models called random forests that are made up of many decision trees. Using these simply conceived models, we were able to make better predictions than we could with logistic regression, judging by the cross-validation ROC AUC score. This is often the case for many real-world problems. Decision trees are robust to a lot of the potential issues that can prevent logistic regression models from good performance, such as non-linear relationships between features and the response variable, and the presence of complicated interactions among features.

Although a single decision tree is prone to overfitting, the random forest ensemble method has been shown to reduce this high-variance issue. Random forests are built by training many trees. The decreased

variance of the ensemble of trees is achieved by increasing the bias of the individual trees in the forest, by only training them on a portion of the available training set (bootstrapped aggregation or bagging), and by only considering a reduced number of features at each node split.

Now that we have tried several different machine learning approaches to modeling the case study data, we found that some work better than others; for example, a random forest with tuned hyperparameters provides the highest average cross-validation ROC AUC score of 0.776, as we saw in *Activity 5, Cross-Validation Grid Search with Random Forest*.

In the next chapter, we'll learn about another type of ensemble method, called gradient boosting, which is often used in conjunction with decision trees. Gradient boosting has yielded some of the best performance of all machine learning models for binary classification use cases. We'll also learn a powerful method for explaining and interpreting the predictions of gradient boosted ensembles of trees, using **SHapely Additive exPlanation (SHAP)** values.